

M9T9

Entrega final reestructurada del ejercicio de análisis de malware

Muestra:

7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be

Familia asociada en fuentes OSINT: Vawtrak / Neverquest

Campo	Valor
Tipo de muestra	PE32 Windows / posible dropper-loader
Arquitectura	x86 / i386
Entorno de análisis	Linux + Ghidra + x32dbg en VM Windows
Objetivo	Caracterizar la muestra, confirmar dropper/persistencia, extraer payload final y recuperar configuración
Fecha de generación	08/06/2026

Índice final propuesto

La entrega se ha reorganizado para seguir una cadena analítica clara: caracterización, estático, validación PE, ofuscación, comportamiento dropper, análisis dinámico, payload final, configuración y conclusiones.

1. Información general de la muestra
2. Triage estático inicial
3. Validación de la estructura PE
4. Análisis de ofuscación y posible empaquetado
5. Análisis funcional estático de la muestra original
6. Confirmación del comportamiento dropper
7. Análisis dinámico
 - 7.1 Preparación del entorno de análisis
 - 7.2 Autocopia y artefactos en disco
 - 7.3 Comprobaciones de marcadores: Xande, Mizuk y Bupra
 - 7.4 Persistencia mediante HKCU\...\Run
 - 7.5 Reserva de memoria y memoria mapeada
 - 7.6 Obtención del payload final reconstruido en memoria
 - 7.7 Carga del payload final en Ghidra
 - 7.8 Análisis de funciones relevantes del payload final
 - 7.9 Análisis de capacidades del payload final con CAPA/FLOSS
 - 7.10 Búsqueda de comunicación C2 y WinINet
 - 7.11 Extracción y análisis del blob de configuración
 - 7.12 Análisis del stage/buffer dinámico adicional
 - 7.13 Técnicas anti-análisis y cierre del proceso
 - 7.14 Limitaciones del análisis dinámico
8. Indicadores de compromiso
9. Conclusiones

Resumen ejecutivo

La muestra M9T9 corresponde a un ejecutable PE32 de Windows asociado en fuentes OSINT a la familia Vawtrak/Neverquest. El análisis muestra un comportamiento compatible con un loader/dropper: la muestra se autocopia, crea persistencia, reconstruye un payload final en memoria y contiene una configuración de red ofuscada que puede recuperarse tras el análisis dinámico.

El hallazgo central es la separación entre tres artefactos: la muestra original, la autocopia persistente Zedyi.exe y el payload final reconstruido en memoria. Zedyi.exe no se considera un payload distinto cuando su hash coincide con el de la muestra original; se trata de la copia usada para persistencia. El payload final, en cambio, se obtiene mediante dump de memoria y contiene la lógica posterior que se analiza en Ghidra.

Artefacto	Clasificación	Interpretación
Muestra original	Loader/dropper	Binario inicial analizado, empaquetado/ofuscado parcialmente.
C:\ProgramData\Dizla\Zedyi.exe	Autocopia	Copia de la muestra original usada para persistencia.
mapped_inner_payload_00110000_size_00016000.bin	Payload final	PE reconstruido en memoria; entry dinámico 0x001166F2.
config_blob.bin	Configuración cifrada/ofuscada	Bloque con 6 entradas descifradas mediante LCG + XOR.
Blob Base64 de 332 caracteres	Datos opacos	Decodifica a 249 bytes de alta entropía; no se confirma como payload adicional.

1. Información general de la muestra

1.1 Primera caracterización

La muestra se identifica como un ejecutable PE32 para Microsoft Windows, arquitectura Intel i386, con subsistema GUI. La presencia de metadatos incoherentes y de nombres de producto aparentemente falsificados refuerza la necesidad de tratar el binario como potencialmente malicioso y ofuscado.

Campo	Valor
SHA-256	7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be
MD5	9724934a3ed3a92b38b89fe7a2e9b6fd
Tipo	PE32 executable for MS Windows
Arquitectura	Intel i386 / x86
Subsistema	Windows GUI
Tamaño	311.296 bytes aprox.
Entry point inicial	0x79C1
Metadatos sospechosos	Company Name: Sopcast; Original File Name: EquippedRobust.exe

1.2 Hashes y footprinting

Los hashes se usan como identificadores estables durante todo el análisis. El SHA-256 coincide con el nombre asignado al archivo en el laboratorio, lo que reduce el riesgo de confusión entre la muestra original, sus copias y los volcados posteriores.

1.3 Consultas externas

Las consultas externas permiten asociar la muestra con Vawtrak/Neverquest. Esta atribución se trata como hipótesis de apoyo: orienta la búsqueda de comunicación HTTP, configuración cifrada, persistencia, carga dinámica y posibles módulos adicionales, pero se valida mediante evidencias propias obtenidas en estático y dinámico.

2. Triage estático inicial

2.1 Identificación PE y arquitectura

El triage confirma que el binario es PE32, little endian, no firmado, con símbolos eliminados y sin overlay detectado. Estas características son compatibles con una muestra preparada para dificultar el análisis estático, aunque no son maliciosas por sí solas.

Indicador	Resultado	Lectura analítica
Formato	PE32	Ejecutable Windows de 32 bits.
Base	0x400000	Base habitual en PE de Windows.
Firmado	No	Ausencia de firma digital.
Stripped	Sí	Sin símbolos útiles para reversing.
Overlay	No	No hay payload anexo de forma trivial al final del archivo.
NX	Sí	No impide carga dinámica si el malware usa VirtualAlloc/VirtualProtect.

2.2 Metadatos, imports y strings iniciales

Los metadatos de versión presentan nombres poco coherentes con software legítimo. La muestra conserva muchas cadenas operativas directas, entre ellas APIs de fichero, recursos, GUI, resolución dinámica y runtime C/C++. Sin embargo, no aparecen de forma directa URLs o indicadores C2 en claro en la muestra original.

2.3 CAPA sobre la muestra original

CAPA detecta capacidades compatibles con dropper/loader: acceso a ficheros, escritura, extracción de recursos, parsing de cabeceras PE, resolución dinámica de APIs, captura de pantalla, descubrimiento de ventanas y terminación de procesos.

2.4 FLOSS sobre la muestra original

FLOSS recupera cadenas estáticas, pero no stack strings ni decoded strings. Esta ausencia no descarta ofuscación: puede indicar que la rutina de decodificación es personalizada, depende del contexto de ejecución o que las cadenas sensibles se encuentran en el payload reconstruido posteriormente.

2.5 Conclusión del triage estático

El triage no permite recuperar directamente la configuración ni un payload completo en disco. Por ello, el análisis debe continuar revisando secciones opacas, funciones de carga y comportamiento dinámico.

3. Validación de la estructura PE

3.1 Secciones

La muestra presenta cinco secciones principales, entre ellas .text, .rdata, .data, .rsrc y .reloc. La presencia de .rsrc y .rdata resulta importante por la hipótesis de dropper: ambas pueden contener configuración, recursos, blobs cifrados o datos de alta entropía.

3.2 Data Directories

La revisión de directorios PE permite identificar la tabla de imports, recursos, relocalaciones y otros elementos estructurales. No se observa un payload PE embebido directamente a través de un directorio obvio, por lo que se descarta una extracción trivial desde recursos.

3.3 Firma digital y Rich Header

La muestra no está firmada digitalmente. La información Rich/Header y los metadatos deben tratarse con cautela, porque pueden ser manipulados o heredados de toolchains legítimas para aparentar normalidad.

4. Análisis de ofuscación y posible empaquetado

4.1 Comprobación de UPX/packer

No se confirma un empaquetado UPX estándar. La ausencia de firma UPX no descarta empaquetado, cifrado o reconstrucción personalizada en memoria.

4.2 Entropía de secciones

La sección .rdata contiene regiones opacas con entropía elevada. Esto explica las alertas heurísticas de herramientas como Detect It Easy, aunque por sí solo no demuestra la existencia de un payload PE embebido.

4.3 Revisión de .rdata y blobs opacos

Se extrae .rdata y se revisan candidatos MZ/PE/PK. Las coincidencias encontradas se descartan como falsos positivos porque no presentan un e_lfanew válido ni cabeceras ZIP/PE coherentes.

4.4 Validación de falsos positivos MZ/PE/PK

Las secuencias MZ y PK localizadas no cumplen la estructura de cabecera esperada. Por tanto, no se puede afirmar que exista un PE o ZIP embebido de forma directa en .rdata.

4.5 Imports sospechosos: GetProcAddress, LoadLibrary, WriteFile, VirtualAlloc

La presencia de APIs de resolución dinámica, memoria y escritura de ficheros orienta la investigación hacia un loader. GetProcAddress y LoadLibrary permiten ocultar capacidades reales; WriteFile y VirtualAlloc son relevantes para autocopia y reconstrucción dinámica.

4.6 Conclusión sobre empaquetado/ofuscación

La muestra presenta ofuscación parcial y datos opacos, pero no un empaquetado estándar fácilmente reconocible. El payload final no se recupera de forma limpia desde el PE inicial; debe obtenerse mediante análisis dinámico.

5. Análisis funcional estático de la muestra original

5.1 Funciones relevantes identificadas

El análisis estático prioriza funciones con referencias a escritura de ficheros, lectura de la propia muestra, carga dinámica de APIs, memoria y parsing PE. Estas zonas actúan como puntos de partida para la depuración dinámica.

5.2 Resolución dinámica de APIs

Las referencias a GetProcAddress y LoadLibrary sugieren que parte de la funcionalidad se resuelve en tiempo de ejecución. Esta técnica dificulta el análisis basado únicamente en imports estáticos.

5.3 Indicios de dropper/loader

Los patrones de lectura/escritura de ficheros, parsing PE, reserva de memoria y llamadas indirectas refuerzan la hipótesis de que la muestra original prepara y ejecuta una etapa adicional.

5.4 Limitaciones del análisis estático inicial

El análisis estático no permite observar la configuración C2 final ni el payload reconstruido. Por este motivo, se pasa a x32dbg y a la captura de memoria durante la ejecución.

6. Confirmación del comportamiento dropper

6.1 Seguimiento de WriteFile/NtWriteFile

En dinámico se observa que la muestra abre su propio archivo, obtiene su tamaño y escribe una copia completa. El valor 0x4C000 coincide con el tamaño de la muestra original, lo que permite confirmar autocopia.

6.2 Autocopia en C:\ProgramData\Dizla\Zedyi.exe

La muestra crea el fichero C:\ProgramData\Dizla\Zedyi.exe. Este artefacto no debe confundirse con el payload final: es una copia persistente de la muestra original.

6.3 Verificación del hash de Zedyi.exe

Al comparar el hash de Zedyi.exe con el de la muestra original se confirma que ambos coinciden. Por tanto, Zedyi.exe es una autocopia, no una nueva etapa diferenciada.

6.4 Diferencia entre autocopia y payload final

Elemento	Origen	Función
Muestra original	Archivo inicial del laboratorio	Loader/dropper.
Zedyi.exe	Creado en C:\ProgramData\Dizla	Autocopia usada para persistencia.
Payload final	Reconstruido en memoria	Código interno ejecutado tras la reconstrucción.

7. Análisis dinámico

7.1 Preparación del entorno de análisis

La muestra se ejecuta en una VM Windows controlada, con x32dbg como depurador principal y herramientas auxiliares como Procmon, Ghidra, CAPA y FLOSS. La VM debe restaurarse o limpiarse entre ejecuciones para evitar persistencia residual.

7.2 Autocopia y artefactos en disco

Durante la ejecución, el malware crea C:\ProgramData\Dizla\Zedyi.exe. Además, realiza comprobaciones de otros ficheros en la misma ruta, lo que sugiere el uso de marcadores de estado o ramas condicionales.

7.3 Comprobaciones de marcadores: Xande, Mizuk y Bupra

Se observan accesos a nombres como Xande, Mizuk y Bupra. La interpretación prudente es que estos ficheros actúan como marcadores o comprobaciones de entorno. Cuando no existen, algunas ramas pueden terminar antes de ejecutar comportamiento posterior.

7.4 Persistencia mediante HKCU\...\Run

Se confirma persistencia en HKCU\Software\Microsoft\Windows\CurrentVersion\Run mediante el valor Sumos, que apunta a C:\ProgramData\Dizla\Zedyi.exe. También se observa una tarea programada sospechosa con nombre similar a MicrosoftWindowsApplicationExperienceProgramDataUpdater, pendiente de validación completa.

Clave: HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Valor: Sumos

Datos: "C:\ProgramData\Dizla\Zedyi.exe"

7.5 Reserva de memoria y memoria mapeada

La muestra crea objetos de memoria mapeada anónimos o pagefile-backed con nombre tipo GUID y tamaño aproximado 0x10000C. El acceso solicitado incluye FILE_MAP_ALL_ACCESS, por lo que el mapping puede ser usado para almacenar, compartir o reconstruir estructuras internas.

API	Parámetros relevantes	Interpretación
CreateFileMappingA	hFile = FFFFFFFF; Size = 0x10000C; Name = GUID	Mapping anónimo nombrado.
MapViewOfFile	DesiredAccess = 0x000F001F; ViewSize = 0x10000C	Vista mapeada con acceso completo.
CreateThread	Parámetro cercano a buffers internos	Posible procesamiento paralelo del mapping.

7.6 Obtención del payload final reconstruido en memoria

La depuración permite salir del bucle de reconstrucción y observar que una función devuelve en EAX la base del payload reconstruido. En una ejecución se identifica EAX = 06CBD5E8 y se guarda una región de 0x1B000 bytes. En otra ejecución más limpia se identifica una imagen mapeada en 0x00110000 con tamaño 0x16000.

```
savedata "C:\Temp\payload_final_06CBD5E8_1B000.bin", 06CBD5E8, 1B000
savedata "C:\Users\usuario\Desktop\mapped_inner_payload_00110000_size_00016000.bin", 00110000, 16000
```

La segunda versión se considera especialmente útil para Ghidra porque conserva una imagen PE coherente con SizeOfImage = 0x16000 y AddressOfEntryPoint = 0x66F2.

7.7 Carga del payload final en Ghidra

El payload se importa como Raw Binary, arquitectura x86:LE:32:default:gcc, base 0x00110000 y longitud 0x16000. Esta base es crítica: permite que la dirección observada dinámicamente, 0x001166F2, coincida en Ghidra con FUN_001166F2.

```
Formato: Raw Binary
Lenguaje: x86:LE:32:default:gcc
Base Address: 0x00110000
Length: 0x16000
Entry dinámico observado: 0x001166F2
RVA interno: 0x001166F2 - 0x00110000 = 0x66F2
```

7.8 Análisis de funciones relevantes del payload final

La función FUN_001166F2 se selecciona porque el loader transfiere la ejecución mediante call eax con EAX = 0x001166F2. No se elige de forma arbitraria: es el punto de entrada dinámico observado en x32dbg.

Función	Rol interpretado
FUN_001166F2	Entrada/dispatcher del payload final; recibe contexto y llama a FUN_0011668C.
FUN_0011668C	Rutina central posterior a la inicialización.
FUN_00116658	Función relacionada con inicialización o preparación de contexto.
FUN_00112FA2	Rutina auxiliar priorizada por proximidad al flujo inicial.
FUN_001137AC / FUN_00113AC2 / FUN_001138CA / FUN_00113EAF	Bloque de funciones internas de procesamiento.
FUN_00117C26 / FUN_00117A6E / FUN_001177E2	Funciones relacionadas con lógica posterior y/o parsing.
FUN_00113988 / FUN_0011853C	Rutinas auxiliares relevantes para revisión manual.

7.9 Análisis de capacidades del payload final con CAPA/FLOSS

Al aplicar herramientas al payload final, el análisis se vuelve más útil que sobre la muestra original: aparecen cadenas y capacidades asociadas a HTTP/HTTPS, POST, cabeceras, wininet.dll, regsvr32.exe y DllRegisterServer. FLOSS no recupera decoded strings, por lo que la configuración debe extraerse manualmente o mediante análisis dinámico.

7.10 Búsqueda de comunicación C2 y WinINet

El payload final contiene referencias a WinINet y cadenas asociadas a comunicación HTTP, pero la ejecución dinámica no siempre llega a realizar comunicación de red observable antes de terminar. Por ello se prioriza la extracción de configuración estática/dinámica del payload.

7.11 Extracción y análisis del blob de configuración

Se localiza un bloque de configuración con contador de entradas en offset 0x204. El contador vale 6. Cada entrada tiene tamaño 0x44: un DWORD de semilla seguido de 0x40 bytes cifrados. El tamaño total del bloque es 0x39E.

```
entry_count = blob[0x204] = 6
size = 0x206 + entry_count * 0x44
size = 0x206 + 6 * 0x44 = 0x39E
Inicio: 06B01AE8
Dumpeo: savedata "C:\Users\usuario\Desktop\config_blob.bin", 06B01AE8, 0000039E
```

El descifrado utiliza un PRNG LCG con la fórmula $state = state * 0x41C64E6D + 0x3039$, y cada byte se recupera mediante XOR con el byte bajo del valor generado.

Entrada	Dominio recuperado
0	ninthclub.com
1	camelcap.com
2	ideagreens.com
3	guesstrade.com
4	castuning.ru
5	mgsmmedia.ru

Blob Base64 adicional

Además de la configuración C2 descifrada, se identifica un blob Base64 de 332 caracteres que decodifica a 249 bytes binarios. Su entropía es 6.956 y no contiene cabeceras conocidas ni strings útiles. Se clasifica como datos opacos o posible configuración cifrada, pero no como payload adicional confirmado.

7.12 Análisis del stage/buffer dinámico adicional

Durante el análisis aparece un bloque dinámico adicional con comportamiento de stage ejecutable. El inicio actúa como trampolín y salta a otra zona de código. Se observan patrones compatibles con acceso al PEB, recorrido de estructuras del loader y resolución manual de módulos/API.

7.12.1 FUN_068704C0

Esta función confirma acceso al PEB mediante fs:[30h] y recorrido de estructuras del loader para localizar módulos cargados. Este comportamiento es típico de resolvedores manuales de APIs que evitan depender de imports estáticos.

7.12.2 FUN_06870770

Esta función actúa como resolvedor dinámico de APIs por hash. Obtiene la base de kernel32, resuelve funciones por parsing de exports y carga módulos como ntdll y advapi32. Esto refuerza el uso de carga dinámica y evasión del análisis estático.

7.13 Técnicas anti-análisis y cierre del proceso

Se observan llamadas y referencias compatibles con anti-análisis: IsDebuggerPresent, GetTickCount, QueryPerformanceCounter, Sleep, RaiseException e IsProcessorFeaturePresent. Sin embargo, debe distinguirse entre anti-debug propio y rutinas estándar de runtime. En el caso analizado, algunas referencias a IsDebuggerPresent aparecen dentro de manejo de fallo /GS y reporte de seguridad.

La ejecución puede finalizar con STATUS_ACCESS_VIOLATION (C0000005) o mediante TerminateProcess tras corrupción de pila detectada. Esto limita el seguimiento dinámico continuo y obliga a combinar snapshots, dumps de memoria y análisis estático del payload reconstruido.

7.14 Limitaciones del análisis dinámico

No siempre se consigue que el proceso alcance la comunicación de red en vivo. Las bases de memoria varían entre ejecuciones, algunos marcadores condicionan ramas y el proceso puede cerrarse antes de completar la lógica. Por ello, los hallazgos se documentan diferenciando evidencia confirmada, indicio y limitación.

8. Indicadores de compromiso

Los siguientes indicadores se derivan del análisis estático y dinámico realizado. Algunos son artefactos confirmados en ejecución; otros corresponden a configuración recuperada del payload final.

Tipo	Indicador	Estado
SHA-256	7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be	Muestra original
MD5	9724934a3ed3a92b38b89fe7a2e9b6fd	Muestra original
Fichero	C:\ProgramData\Dizla\Zedyi.exe	Autocopia / persistencia
Registro	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Sumos	Persistencia confirmada
Tarea	\MicrosoftWindowsApplicationExperienceProgramDataUpdater	Sospechosa / pendiente de validación completa
Marcador	C:\ProgramData\Dizla\Xande	Check de existencia
Marcador	C:\ProgramData\Dizla\Mizuk	Check de existencia
Marcador	C:\ProgramData\Dizla\Bupra	Check de existencia
Dominio	ninthclub.com	Configuración C2 recuperada
Dominio	camelcap.com	Configuración C2 recuperada
Dominio	ideagreens.com	Configuración C2 recuperada
Dominio	guesstrade.com	Configuración C2 recuperada
Dominio	castuning.ru	Configuración C2 recuperada
Dominio	mgsmedia.ru	Configuración C2 recuperada

9. Conclusiones

- La muestra inicial se comporta como un loader/dropper de Windows de 32 bits asociado a Vawtrak/Neverquest en fuentes externas.
- El análisis estático inicial muestra ofuscación parcial y datos opacos, pero no permite extraer directamente el payload final ni la configuración C2.
- La muestra se autocopia como C:\ProgramData\Dizla\Zedyi.exe. Esta autocopia mantiene el mismo hash que el binario original, por lo que no se clasifica como una segunda etapa distinta.
- La persistencia se confirma mediante HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Sumos, que apunta a Zedyi.exe.
- Durante la ejecución se crean objetos de memoria mapeada y regiones privadas ejecutables/escribibles, compatibles con reconstrucción dinámica de código.
- El payload final se reconstruye en memoria y puede ser dumpeado como mapped_inner_payload_00110000_size_00016000.bin o payload_final_06CBD5E8_1B000.bin según la ejecución.
- El destino dinámico observado, EAX = 0x001166F2 antes de call eax, justifica analizar FUN_001166F2 como punto de entrada del payload final en Ghidra.
- El payload final contiene capacidades de red y carga dinámica, con referencias a WinINet, HTTP/HTTPS, POST, regsvr32.exe y DllRegisterServer.
- Se recupera una configuración C2 cifrada/ofuscada con 6 dominios mediante un algoritmo basado en LCG + XOR.
- Existe un blob Base64 adicional de alta entropía que no se confirma como payload; se clasifica como dato opaco o posible configuración auxiliar.
- El análisis dinámico se ve limitado por cierres, excepciones y posibles técnicas anti-análisis; por ello se combinan dumps de memoria, Ghidra, CAPA, FLOSS y x32dbg.

Conclusión final: la muestra actúa como primera etapa de un malware bancario/loader, se instala mediante autocopia y persistencia de usuario, reconstruye una segunda etapa en memoria y contiene configuración C2 recuperable mediante descifrado específico. La separación clara entre muestra original, autocopia, payload final y blob de configuración es esencial para interpretar correctamente el flujo del malware.

Anexo A. Comandos y puntos de control recomendados

A.1 Hashes

```
sha256sum 7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be
md5sum 7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be
```

A.2 Limpieza de persistencia en VM

```
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Sumos /f
schtasks /delete /tn "\MicrosoftWindowsApplicationExperienceProgramDataUpdater" /f
del C:\ProgramData\Dizla\Zedyi.exe
```

A.3 Breakpoints mínimos para payload reconstruido

```
bp kernel32.CreateFileMappingA
bp kernel32.CreateFileMappingW
bp kernel32.MapViewOfFile
bp ntdll.NtCreateSection
bp ntdll.NtMapViewOfSection
```

A.4 Carga en Ghidra del payload limpio

```
Format: Raw Binary
Language: x86:LE:32:default:gcc
Base Address: 0x00110000
Length: 0x16000
```

A.5 Fórmula de descifrado de configuración

```
state = (state * 0x41C64E6D + 0x3039) & 0xFFFFFFFF
plain_byte = encoded_byte ^ (state & 0xFF)
```